GenAI Interview Guide

Part 2 — RAG Engineering + Advanced RAG

30 questions with follow-ups · 3–5 YOE · Interview Prep 2025

Contents

Section 1 — RAG Engineering [Most Asked in AI Interviews]

Q01. What is RAG?

Frequently Asked

RAG (Retrieval Augmented Generation) — a technique that combines retrieval from an external knowledge base with LLM text generation. Instead of relying solely on the model's trained knowledge, RAG fetches relevant documents at inference time and injects them into the prompt as context.

Why RAG exists — three core LLM limitations it solves:

1. Knowledge cutoff: LLMs don't know about events after training. RAG fetches current data.
2. Hallucination: LLMs generate plausible but wrong answers for things they don't know. RAG grounds responses in retrieved facts.
3. Private data: LLMs aren't trained on your company's internal documents. RAG provides them at query time.

Simple flow:

User query → retrieve relevant documents → inject into prompt → LLM generates grounded answer

Real-world use cases: company knowledge bases, legal document Q&A, codebase search, customer support, medical records analysis.

⭐ **Key Insight:** RAG = give the LLM a textbook to read before answering, rather than making it rely on memory alone.

↳ Follow-up: When should you NOT use RAG?

Avoid RAG when: (1) All needed knowledge fits comfortably in context window and rarely changes — just include it directly. (2) Low-latency is critical and retrieval adds unacceptable delay. (3) The task is creative/generative (writing, summarisation) not factual. (4) Your corpus is too small to benefit from vector search — just use keyword search or include everything. RAG adds complexity — don't use it unless you need it.

Q02. Explain the complete RAG pipeline.

Frequently Asked

RAG has two main phases:

INDEXING PHASE (offline, done once or periodically):

1. Load documents (PDFs, web pages, databases, markdown files)
2. Chunk documents into smaller pieces (e.g. 512 tokens each)
3. Embed each chunk using an embedding model → dense vector
4. Store vectors + metadata in a vector database

RETRIEVAL + GENERATION PHASE (online, every query):

1. User submits query

2. Embed query using the SAME embedding model
3. Vector similarity search → top-K most similar chunks retrieved
4. (Optional) Rerank retrieved chunks for better precision
5. Inject retrieved chunks + query into LLM prompt
6. LLM generates answer grounded in retrieved context
7. Return answer (optionally with source citations)

```
# Complete minimal RAG pipeline
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

# ── INDEXING (offline) ──────────────────────────
# 1. Load
loader = PyPDFLoader("company_handbook.pdf")
documents = loader.load()

# 2. Chunk
splitter = RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=50)
chunks = splitter.split_documents(documents)

# 3+4. Embed and store
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(chunks, embeddings)

# ── RETRIEVAL + GENERATION (online) ─────────────
# 5-7. Retrieve, inject, generate
qa_chain = RetrievalQA.from_chain_type(
    llm=OpenAI(),
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5}),
)
answer = qa_chain.run("What is the leave policy?")
```

↳ Follow-up: What is the most common failure point in a RAG pipeline?

Retrieval failure — the right document exists in the corpus but the retrieval step doesn't surface it. Causes: (1) Bad chunking — answer split across chunk boundaries. (2) Semantic mismatch — query language doesn't match document language. (3) Wrong embedding model. (4) Not enough chunks retrieved (K too small). 80% of RAG quality problems are retrieval problems, not generation problems.

Q03. Indexing vs retrieval phase?

Frequently Asked

INDEXING PHASE (offline preprocessing):
— When it runs: once when documents are added/updated
— What it does: load → chunk → embed → store in vector DB
— Cost: high compute (embedding millions of chunks), run once
— Goal: prepare a searchable index
RETRIEVAL PHASE (online, every query):
— When it runs: on every user query
— What it does: embed query → vector search → rerank → inject into prompt
— Cost: low compute but latency-sensitive (must be fast)
— Goal: find the most relevant chunks for this specific query
Key distinction: indexing is a BATCH job. Retrieval is a REAL-TIME operation.
Why the split matters in production:
— Indexing can be slow (acceptable) — retrieval must be fast (< 200ms target)
— Indexing can be parallelised — retrieval is on the critical path
— Re-indexing is expensive — change chunking strategy carefully

↳ Follow-up: What triggers re-indexing?

Re-indexing needed when: (1) New documents added to corpus. (2) Existing documents updated (need to delete old chunks + add new ones). (3) Chunking strategy changes. (4) Embedding model changed (all existing embeddings become incompatible

— full re-index required). Model changes are most expensive — choose your embedding model carefully and don't change it lightly.

Q04. What are chunking strategies?

Frequently Asked

Chunking = splitting documents into pieces that are the right size for retrieval. The most impactful decision in RAG system design.

Common strategies:

1. Fixed-size chunking: split every N tokens/characters, with overlap
   — Simple, fast, predictable. Overlap prevents answers from being split across chunks.
   — Problem: splits mid-sentence, mid-paragraph — loses semantic coherence.
2. Sentence chunking: split at sentence boundaries
   — Preserves semantic units. Good for Q&A where answers are sentence-sized.
3. Recursive character splitting: try splitting by paragraph, then sentence, then word
   — LangChain's RecursiveCharacterTextSplitter uses this. Best default choice.
4. Semantic chunking: embed sentences, split when cosine similarity drops
   — Groups semantically related sentences together. High quality but slower.
5. Document-structure-aware: respect headers, sections, bullet points (Markdown, HTML)
   — Best for structured documents. Preserve document hierarchy.
6. Parent-child chunking: index small chunks for retrieval, but return larger parent chunks for context
   — Best of both worlds — precise retrieval + sufficient context for generation.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Default strategy — good starting point
splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,       # target chunk size in characters
    chunk_overlap=50,     # overlap between consecutive chunks
    separators=["

", "
", ". ", " ", ""],  # try these in order
)

chunks = splitter.split_text(document_text)

# Semantic chunking (requires embedding each sentence — slower)
from langchain_experimental.text_splitter import SemanticChunker
from langchain.embeddings import OpenAIEmbeddings

semantic_splitter = SemanticChunker(
    OpenAIEmbeddings(),
    breakpoint_threshold_type="percentile"  # split on semantic drops
)
semantic_chunks = semantic_splitter.split_text(document_text)
```

💡 **Tip:** Start with RecursiveCharacterTextSplitter, chunk_size=512, overlap=50. Evaluate retrieval quality. Tune from there.

↳ Follow-up: What is chunk overlap and why is it important?

Chunk overlap = N tokens shared between consecutive chunks. Example: chunk 1 ends at token 512, chunk 2 starts at token 462 (50-token overlap). This ensures answers that straddle a chunk boundary are still findable. Without overlap, a question whose answer spans the end of chunk 1 and start of chunk 2 would return an incomplete answer. Rule of thumb: overlap = 10-15% of chunk size.

Q05. Why does chunk size matter?

Frequently Asked

Chunk size controls the precision-context tradeoff — one of the most important tuning decisions in RAG.

Too small chunks (1000 tokens):

— Lots of context: LLM has plenty of information

— Problem: low precision — chunk contains lots of irrelevant text

— Problem: top-K retrieval retrieves fewer unique documents (large chunks use up K slots)

— Problem: LLM gets confused with too much irrelevant context in one chunk
Sweet spot: 256–512 tokens for most use cases.
Rule of thumb by use case:
— Q&A over short facts: 128–256 tokens
— Technical documentation: 512–1024 tokens
— Legal/medical long-form: 1024 tokens + parent-child chunking

⚠️ **Interview trap:** *There is no universally optimal chunk size. It depends on your document type, query type, and embedding model. Always evaluate empirically.*

↳ Follow-up: How do you empirically determine the right chunk size?

Build a small evaluation set of 50-100 question-answer pairs from your documents. For each chunk size candidate (128, 256, 512, 1024), run retrieval and measure: (1) Does the correct chunk appear in top-5 results? (Recall@5) (2) Is the answer faithfully generated from retrieved context? (Faithfulness). Pick the chunk size that maximises both metrics for your specific document type.

Q06. What are vector databases?

Frequently Asked

Vector databases are purpose-built databases for storing, indexing, and querying high-dimensional vectors (embeddings). They enable approximate nearest-neighbour (ANN) search at scale — finding the most similar vectors to a query vector extremely fast.
Core operations:
— Insert: store vector + metadata + document chunk ID
— Query: given a query vector, find top-K most similar stored vectors
— Delete/Update: remove or replace vectors (needed when documents change)
— Filter: combine vector similarity with metadata filters
Popular vector DBs:
— Pinecone: fully managed, cloud-native, production-grade
— Weaviate: open-source, multi-modal, strong hybrid search
— ChromaDB: open-source, great for development/prototyping
— Qdrant: open-source, Rust-based, very fast, good filtering
— pgvector: Postgres extension — add vector search to existing Postgres
— FAISS: Facebook's in-memory library (not a full DB, no persistence)

```
# ChromaDB example (local development)
import chromadb
from chromadb.utils import embedding_functions

client = chromadb.Client()
ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key="your-key",
    model_name="text-embedding-3-small"
)

collection = client.create_collection("my_docs", embedding_function=ef)

# Insert chunks
collection.add(
    documents=["chunk text 1", "chunk text 2", "chunk text 3"],
    metadatas=[{"source": "doc1.pdf", "page": 1},
               {"source": "doc1.pdf", "page": 2},
               {"source": "doc2.pdf", "page": 1}],
    ids=["chunk_1", "chunk_2", "chunk_3"]
)

# Query — returns top 3 most similar chunks
results = collection.query(
    query_texts=["What is the leave policy?"],
    n_results=3,
    where={"source": "doc1.pdf"}  # optional metadata filter
)
```

↳ Follow-up: What indexing algorithm does vector databases use internally?

Most use HNSW (Hierarchical Navigable Small World) — a graph-based ANN algorithm. It builds a multi-layer graph where each node connects to its nearest neighbours. Query = traverse the graph top-down to find approximate nearest neighbours.

HNSW gives excellent recall at high speed. Tradeoff: higher memory usage than flat indexes. FAISS also offers IVF (inverted file index) and PQ (product quantisation) for billion-scale vectors.

Q07. Dense retrieval vs BM25?

Frequently Asked

Two fundamentally different retrieval approaches:
BM25 (Sparse/Keyword retrieval):
— Classic TF-IDF based algorithm. Searches for exact keyword matches.
— Fast, no ML needed, great for exact term matching
— Works well when query terms appear in documents verbatim
— Fails for synonyms: "automobile" won't match "car"
— Still used: Elasticsearch, OpenSearch, Solr all use BM25
Dense retrieval (Semantic/Vector retrieval):
— Embed query and documents into dense vectors. Search by cosine similarity.
— Understands semantics: "automobile" and "car" have similar embeddings
— Handles paraphrasing, synonyms, concepts
— Requires embedding model (compute overhead)
— Fails for exact matches: "API-123-XYZ" may not match well on dense search
When BM25 wins: exact term matches, product codes, serial numbers, proper nouns
When dense wins: semantic questions, paraphrased queries, conceptual search

↳ Follow-up: Which is better for production RAG?

Neither alone — use HYBRID. Most production RAG systems combine BM25 + dense retrieval. BM25 catches exact keyword matches that dense retrieval misses. Dense retrieval catches semantic matches that BM25 misses. Hybrid consistently outperforms either alone. See Q8 on hybrid search.

Q08. What is hybrid search?

Frequently Asked

Hybrid search combines dense (vector) retrieval and sparse (BM25/keyword) retrieval to get the best of both worlds. Results from both systems are merged and re-ranked.
How it works:
1. Run dense retrieval → top-K dense results with similarity scores
2. Run sparse (BM25) retrieval → top-K keyword results with BM25 scores
3. Combine/fuse both result sets using a fusion algorithm
4. Return unified top-K results
Fusion algorithms:
— Reciprocal Rank Fusion (RRF): score = Σ 1/(rank + k). Simple, robust, no score normalisation needed.
— Weighted sum: normalise scores from both systems, compute weighted average
— Learning-to-rank: train a model to rerank the combined results
Why hybrid consistently wins:
— Dense retrieval finds semantically similar but not keyword-matching docs
— BM25 finds exact matches that dense retrieval ranks too low
— Together they cover each other's blind spots

```
# Hybrid search with Qdrant
from qdrant_client import QdrantClient
from qdrant_client.models import SparseVector, SparseIndexParams

client = QdrantClient(":memory:")

# Query with both dense and sparse vectors
results = client.query_points(
    collection_name="my_docs",
    prefetch=[
        # Dense retrieval
        {"query": dense_query_vector, "using": "dense", "limit": 20},
        # Sparse (BM25-like) retrieval
        {"query": SparseVector(indices=sparse_indices, values=sparse_values),
         "using": "sparse", "limit": 20},
```

```
    ],
    # Fuse with Reciprocal Rank Fusion
    query={"fusion": "rrf"},
    limit=5  # return top 5 after fusion
)

# RRF formula: score = sum(1 / (rank + 60)) for each result list
```

💡 **Tip:** Reciprocal Rank Fusion (RRF) is the go-to fusion algorithm — simple, parameter-free, and works well in practice.

Q09. What are rerankers?

Frequently Asked

Rerankers are a second-stage retrieval step that re-scores the top-K retrieved chunks using a more accurate (but slower) cross-encoder model, before passing results to the LLM.

Two-stage pipeline:

Stage 1 — Bi-encoder (fast): embed query + all chunks separately, cosine similarity. Used for initial retrieval (top 20-50 candidates).

Stage 2 — Cross-encoder (accurate): takes (query, chunk) PAIR as input, outputs relevance score. More accurate but O(N) per query.

Why not use cross-encoder for everything?

Cross-encoders can't pre-compute chunk representations (they need the query). So they can't do fast vector search. They're used to RERANK a small set of candidates returned by the fast first stage.

Popular rerankers:

— Cohere Rerank API: managed, state-of-the-art

— BGE-Reranker (Hugging Face): open source, good quality

— Cross-Encoder/ms-marco (Sentence Transformers): well-tested

— Flashrank: fast, lightweight reranker

```
# Two-stage retrieval with reranking
import cohere
from your_vectordb import retrieve_top_k

co = cohere.Client("your-api-key")

query = "What is the leave policy for contractors?"

# Stage 1: fast vector retrieval — top 20 candidates
candidates = retrieve_top_k(query, k=20)
candidate_texts = [c.text for c in candidates]

# Stage 2: rerank top 20 → return top 5
rerank_results = co.rerank(
    model="rerank-english-v3.0",
    query=query,
    documents=candidate_texts,
    top_n=5  # return top 5 after reranking
)

# Use reranked top 5 for LLM context
final_chunks = [candidates[r.index] for r in rerank_results.results]
```

↳ Follow-up: How much does reranking improve retrieval quality?

Typically 5-15% improvement in NDCG and recall metrics in benchmarks. In production, the impact is higher for ambiguous or complex queries. Reranking is most valuable when: (1) your initial retrieval has high recall but low precision (many candidates, hard to rank), (2) queries are complex multi-part questions, (3) you have budget for ~100ms extra latency.

Q10. What causes hallucinations in RAG systems?

Frequently Asked

RAG-specific hallucination causes are different from vanilla LLM hallucinations:

1. Retrieval failure (most common): wrong chunks retrieved → LLM has no correct context → generates from memory → hallucinates. The retrieval problem masquerades as a generation problem.

2. Context not used: LLM ignores retrieved context and uses trained knowledge instead. Happens when context contradicts the model's strong prior beliefs.
3. Conflicting context: retrieved chunks contradict each other → LLM tries to reconcile and hallucinates a synthesis.
4. Insufficient context: answer requires connecting information from multiple chunks but only one was retrieved.
5. Lost-in-the-middle: relevant chunk retrieved but buried in the middle of a long context → LLM attends to start/end, misses it.
6. Prompt injection in documents: malicious content in indexed documents can redirect LLM behaviour.

⚠️ **Interview trap:** *Don't blame the LLM first. Always inspect retrieved chunks before blaming generation. Most RAG quality issues are retrieval problems, not model problems.*

↳ Follow-up: How do you debug whether a hallucination is a retrieval problem or a generation problem?

Retrieval inspection test: manually check what chunks were retrieved for the failing query. Ask: "Is the correct answer actually in the retrieved context?" If YES → generation problem (LLM ignored context or reasoned incorrectly). If NO → retrieval problem (fix chunking, embedding, or add more documents). 80% of RAG hallucinations are retrieval problems.

Q11. How do you evaluate RAG systems?

Frequently Asked

RAG evaluation has TWO components — retrieval quality and generation quality:
RETRIEVAL METRICS:
— Context Precision: of retrieved chunks, what fraction are actually relevant?
— Context Recall: of all relevant information needed to answer, what fraction was retrieved?
— Hit Rate / Recall@K: does the correct chunk appear in top-K results?
— MRR (Mean Reciprocal Rank): how highly is the correct chunk ranked?
GENERATION METRICS:
— Faithfulness: is the generated answer supported by the retrieved context? (0-1)
— Answer Relevance: does the answer address the question? (0-1)
— Answer Correctness: is the answer factually correct vs ground truth?
Frameworks:
— RAGAS: open-source RAG evaluation framework. Computes all above metrics automatically using an LLM-as-judge.
— DeepEval: comprehensive LLM testing framework
— TruLens: observability + evaluation for LLM applications

```
# RAGAS evaluation example
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_recall,
    context_precision,
)
from datasets import Dataset

# Your RAG output data
data = {
    "question": ["What is the leave policy?"],
    "answer": ["Employees get 20 days of paid leave per year."],
    "contexts": [["Context chunk 1...", "Context chunk 2..."]],  # retrieved chunks
    "ground_truth": ["The annual leave entitlement is 20 days."],
}
dataset = Dataset.from_dict(data)

results = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy, context_recall, context_precision]
)
print(results)
# faithfulness: 0.92 | answer_relevancy: 0.88 | context_recall: 0.85
```

↳ Follow-up: What is faithfulness vs answer correctness?

Faithfulness: is the answer SUPPORTED BY the retrieved context? (Does the answer make claims that aren't in the context?) Measures hallucination within RAG. Answer Correctness: is the answer FACTUALLY TRUE? Compares against ground truth. A faithful answer can still be incorrect if the retrieved context was wrong. You want BOTH high.

Q12. What is metadata filtering?

Moderately Asked

Metadata filtering = combining vector similarity search with structured attribute filters, so you only search within a relevant subset of documents.

Why needed: without filtering, a query about "Q3 2024 sales" might retrieve chunks from Q1, Q2, or competitor analysis documents that are semantically similar but contextually wrong.

Common metadata fields to filter on:

— Document source: "only search within docs tagged as HR policy"
— Date range: "only retrieve documents from the last 6 months"
— Department/category: "only search within engineering documentation"
— User access level: "only retrieve documents this user is allowed to see"
— Language: "only return English documents"

Pre-filtering vs post-filtering:

— Pre-filtering: filter BEFORE vector search (faster, fewer vectors to search)
— Post-filtering: run vector search on all, then filter results (more accurate but slower)

```
# Metadata filtering in ChromaDB
results = collection.query(
    query_texts=["What is the parental leave policy?"],
    n_results=5,
    where={
        "$and": [
            {"department": {"$eq": "HR"}},
            {"year": {"$gte": 2024}},
        ]
    }
)

# Metadata filtering in Pinecone
index.query(
    vector=query_embedding,
    top_k=5,
    filter={
        "department": {"$eq": "HR"},
        "year": {"$gte": 2024},
    },
    include_metadata=True
)
```

💡 **Tip:** Always store rich metadata at indexing time. You can't filter on fields you didn't index. Common fields: source_file, date, department, author, document_type, version.

Q13. RAG vs long-context models?

Frequently Asked

With models like Gemini 1.5 Pro (1M tokens) and Claude 3.5 (200K tokens), a natural question arises: why not just put all documents in the context window instead of doing RAG?

Long-context models win when:

— Corpus fits in context window (< 200K tokens)
— Content needs to be read holistically (cross-referencing many parts)
— Latency is acceptable (filling 200K context is slow)
— You can afford the cost (long contexts = expensive)
— Documents need to be read in full order/structure

RAG wins when:

— Corpus is too large for context window (millions of documents)
— You need fresh/updated information (new docs added daily)
— Cost matters: retrieve 3-5 chunks (cheap) vs 200K tokens (expensive)
— Latency matters: small context = faster generation
— You need source citations and traceability
— Access control: different users see different document subsets

In practice: most production systems use RAG. Long context is used for specific analytical tasks.

↳ Follow-up: Will RAG become obsolete as context windows grow to 10M tokens?

No — for two reasons: (1) Cost and latency scale with context size. Even with 10M token windows, processing them is expensive and slow. RAG is more efficient. (2) RAG provides access control, freshness, and citations that long-context cannot. You can't give every user access to every document in a 10M context. RAG with selective retrieval remains the architecture of choice for enterprise knowledge systems.

Q14. What is semantic search?

Frequently Asked

Semantic search = finding documents that are CONCEPTUALLY related to a query, even if they don't share exact keywords.

Traditional keyword search: looks for exact word matches. "car accident" won't find "vehicle collision".

Semantic search: embeds query and documents into vector space. Similar concepts are close together. "car accident" and "vehicle collision" have similar embeddings → semantic search finds both.

How it works:

1. Embed all documents with an embedding model → vectors stored in vector DB
2. Embed the user query with the same model
3. Find stored vectors closest to query vector (cosine similarity or dot product)
4. Return documents corresponding to closest vectors

Semantic search IS the retrieval step in RAG. When people say "vector search" or "embedding search" they mean semantic search.

Applications: document search, product search, code search, image search (using CLIP embeddings).

↳ Follow-up: What makes a good embedding model for semantic search?

Key factors: (1) Domain match — models trained on similar content to yours perform better (e.g. legal-bert for legal docs). (2) Dimensionality — higher dims (1536 vs 384) generally better but more expensive. (3) Max sequence length — model must handle your chunk size without truncation. (4) Speed — inference time matters at scale. Popular choices: text-embedding-3-small (OpenAI), BGE-Large (Hugging Face, open source), Cohere Embed v3.

Q15. What are ingestion pipelines?

Moderately Asked

Ingestion pipeline = the automated data processing workflow that takes raw documents and transforms them into indexed, searchable chunks in your vector database.

Components:

1. Document loaders: read from PDF, Word, HTML, databases, APIs, S3, Confluence, Notion
2. Pre-processing: clean text (remove headers/footers, fix encoding), deduplicate
3. Chunking: split into appropriate-sized pieces
4. Metadata extraction: extract date, author, source, category from document
5. Embedding: convert each chunk to vector using embedding model
6. Upsertion: insert/update vectors in vector DB (handle duplicates gracefully)
7. Error handling + retry: handle failed documents without stopping the pipeline

Ingestion modes:

— Batch: process all documents at once (initial load or full re-index)

— Incremental: only process new/changed documents (daily or real-time)

— Real-time: trigger indexing immediately when document is created/updated

```
# LlamaIndex ingestion pipeline
from llama_index.core.ingestion import IngestionPipeline
from llama_index.core.node_parser import SentenceSplitter
from llama_index.core.extractors import TitleExtractor, QuestionsAnsweredExtractor
from llama_index.embeddings.openai import OpenAIEmbedding

pipeline = IngestionPipeline(
    transformations=[
        SentenceSplitter(chunk_size=512, chunk_overlap=50),
        TitleExtractor(),                   # extract document title as metadata
        QuestionsAnsweredExtractor(),        # generate Q's this chunk answers
        OpenAIEmbedding(model="text-embedding-3-small"),
    ],
    vector_store=your_vector_store,
)
```

```
# Run pipeline on documents
nodes = await pipeline.arun(documents=raw_documents, num_workers=4)
```

💡 **Tip:** Add document hash to metadata during ingestion. On re-ingestion, skip documents whose hash hasn't changed — avoids re-embedding unchanged content and saves cost.

Section 2 — Advanced RAG [Senior Level]

Q16. What is Graph RAG?

Frequently Asked

Graph RAG = augmenting retrieval with a knowledge graph built from the document corpus, enabling multi-hop reasoning and relationship traversal.
Standard RAG limitation: retrieves isolated text chunks. Struggles with questions requiring connecting information across multiple documents or understanding relationships between entities.
Graph RAG approach (Microsoft's implementation):
1. Extract entities and relationships from documents (using LLM)
2. Build a knowledge graph: nodes = entities, edges = relationships
3. Create hierarchical summaries of graph communities (clusters)
4. At query time: traverse graph to find related entities + retrieve relevant summaries
Strengths:
— Multi-hop reasoning: "Who works for the company that acquired Startup X?"
— Relationship queries: "What are all the dependencies of this service?"
— Global synthesis: questions that require understanding of the entire corpus
Weaknesses:
— High indexing cost (LLM extraction of every entity)
— Graph maintenance complexity when documents change
— Overkill for simple Q&A use cases

↳ Follow-up: When would you choose Graph RAG over standard RAG?

Choose Graph RAG when queries require multi-hop reasoning across entities (e.g. org charts, code dependency graphs, research citation networks, supply chains). Standard RAG when queries are self-contained within single document sections. Graph RAG indexing costs 5-10x more than standard RAG — only worth it for relationship-heavy domains.

Q17. Parent-child retrieval vs recursive retrieval?

Frequently Asked

Both are strategies to solve the precision-context tradeoff in chunking.
Parent-child retrieval:
— Index SMALL child chunks for retrieval (high precision, finds the exact passage)
— But return the LARGE parent chunk to the LLM (more context for better generation)
— Example: child = 128 tokens, parent = 512 tokens
— Implementation: store parent ID in child metadata, on retrieval fetch parent by ID
Recursive retrieval (small-to-big):
— Build a hierarchy: summaries → sections → paragraphs → sentences
— Retrieve summaries first, then recursively drill down into relevant sections
— Useful for large documents where broad context matters first
When to use each:
— Parent-child: when answer is precise but needs surrounding context. Good default.
— Recursive: when document structure is hierarchical (reports, books, legal docs)
Example parent-child:
User asks: "What is the penalty for late submission?"
→ Child chunk "penalty for late submission is $500 per day" retrieved (precise)
→ Parent chunk (entire contract section on penalties) returned to LLM (full context)

```
# Parent-child chunking with LlamaIndex
from llama_index.core.node_parser import (
    HierarchicalNodeParser,
    get_leaf_nodes,
    get_root_nodes,
)
from llama_index.core.retrievers import AutoMergingRetriever
from llama_index.core import VectorStoreIndex

# Create hierarchical chunks
parser = HierarchicalNodeParser.from_defaults(
    chunk_sizes=[2048, 512, 128]  # parent → child → grandchild
)
```

```
nodes = parser.get_nodes_from_documents(documents)

# Index only leaf (smallest) nodes for retrieval
leaf_nodes = get_leaf_nodes(nodes)
index = VectorStoreIndex(leaf_nodes)

# AutoMergingRetriever: if enough siblings retrieved, return parent
retriever = AutoMergingRetriever(
    index.as_retriever(similarity_top_k=10),
    storage_context,
    verbose=True
)
```

💡 **Tip:** Parent-child retrieval is one of the biggest quality improvements you can make to a basic RAG system. Implement it early.

Q18. What is contextual compression?

Frequently Asked

Contextual compression = compressing or filtering retrieved chunks to extract ONLY the parts relevant to the query, before passing to the LLM.

Problem it solves: retrieved chunks often contain a lot of irrelevant content. A 512-token chunk retrieved for a specific question might only have 2-3 sentences actually relevant to the answer. The remaining 490 tokens dilute the context and can confuse the LLM.

How it works:

1. Retrieve top-K chunks as usual
2. For each chunk, use an LLM (or smaller model) to extract only the relevant sentences
3. Pass the compressed/filtered context to the main LLM for generation

Methods:

— LLM extraction: "Given this context and this question, extract only the relevant sentences."

— Sentence-level filtering: embed each sentence separately, keep only sentences similar to query

— Document compressors: LLMChainFilter (keep/discard), LLMChainExtractor (extract relevant text)

Tradeoff: adds ~100-200ms latency for the compression step. Worth it when chunks are large and context window is a bottleneck.

```
# Contextual compression in LangChain
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor

# Base retriever
base_retriever = vectorstore.as_retriever(search_kwargs={"k": 10})

# Compression: LLM extracts only relevant parts of each chunk
compressor = LLMChainExtractor.from_llm(llm)

# Combined retriever: retrieve 10, compress each, return top 5
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=base_retriever
)

compressed_docs = compression_retriever.get_relevant_documents(
    "What is the late submission penalty?"
)
# Each doc now contains only the relevant extracted sentences
```

↳ Follow-up: What's the difference between contextual compression and reranking?

Reranking: REORDERS the retrieved chunks, discarding lower-ranked ones. All kept chunks are still full-length. Contextual compression: EXTRACTS only relevant sentences from each chunk — reduces chunk content. Use both together for best results: rerank → take top-5 → compress each → pass to LLM.

Q19. Explain retrieval drift.

Moderately Asked

Retrieval drift = when the retrieved documents gradually become less relevant to queries over time, even though the retrieval system hasn't changed.
Causes:
1. Data distribution shift: new queries arrive that don't match the language/topics of indexed documents
2. Embedding model staleness: embedding model trained on old data; new terminology not in its vocabulary
3. Corpus staleness: documents become outdated but are still retrieved as top results
4. Query style change: users start asking questions differently than when the system was designed
Signs of retrieval drift:
— Faithfulness scores dropping over time
— Users reporting answers feel "off" or "outdated"
— Context precision metrics declining gradually
Mitigations:
— Monitor retrieval quality metrics continuously (context precision, hit rate)
— Regular re-indexing with updated documents
— Periodic evaluation with fresh query samples
— Embedding model versioning — track when model was last updated
— Query logging — analyse query patterns to detect distribution shift early

↳ Follow-up: How do you detect embedding drift in production?

Track: (1) Mean cosine similarity of top-1 retrieval over time — if dropping, corpus-query alignment is worsening. (2) Context precision score on a fixed evaluation set — run weekly, alert if drops > 5%. (3) User feedback signals (thumbs up/down, follow-up questions that indicate answer was wrong). (4) Query cluster analysis — are new query clusters appearing that have low retrieval quality?

Q20. What is query rewriting?

Frequently Asked

Query rewriting = transforming the user's raw query into a better form for retrieval before searching the vector database.
Why needed: users ask questions in natural language that may not match how information is stored in documents. "Can you tell me about the company's time off rules?" → poor retrieval. "Paid leave policy annual entitlement" → good retrieval.
Techniques:
1. HyDE (Hypothetical Document Embeddings):
   — Generate a hypothetical ANSWER to the query using LLM, even if wrong
   — Embed the hypothetical answer instead of the question
   — Answers and documents are in the same language space → better match
2. Query expansion:
   — Generate multiple variations/sub-queries from the original
   — Run retrieval for each variation, merge and deduplicate results
   — Catches documents that match one phrasing but not another
3. Step-back prompting:
   — Convert specific query to a more general one for broader retrieval
   — "What medications interact with ibuprofen?" → "What are ibuprofen interactions and contraindications?"
4. Multi-query retrieval:
   — Break complex multi-part question into sub-questions
   — Retrieve separately for each, merge results

```
# HyDE - Hypothetical Document Embeddings
from langchain.chains import HypotheticalDocumentEmbedder
from langchain.prompts import PromptTemplate

# Step 1: Generate hypothetical answer to the query
hyde_prompt = PromptTemplate(
    input_variables=["question"],
    template="Write a short paragraph that would answer: {question}"
)

# The hypothetical answer (even if wrong) is closer to real document language
hypothetical_answer = llm.predict(hyde_prompt.format(
    question="What is the parental leave policy?"
))
# → "Employees are entitled to 26 weeks of parental leave..."
```

```
# Step 2: Embed the hypothetical answer instead of the question
query_embedding = embedder.embed(hypothetical_answer)

# Step 3: Use this embedding for vector search
results = vectorstore.similarity_search_by_vector(query_embedding, k=5)
```

💡 **Tip:** HyDE often gives 5-15% retrieval improvement with minimal code. It's one of the easiest advanced RAG improvements to implement.

Q21. What is multi-hop retrieval?

Frequently Asked

Multi-hop retrieval = answering questions that require finding and connecting information from MULTIPLE separate documents through a chain of reasoning steps.

Single-hop: "Who is the CEO of company X?" → one retrieval, one chunk answers it.

Multi-hop: "What company did the CEO of company X previously work at, and what did that company do?" → requires finding CEO, then finding their history, then finding the other company's description.

Approaches:

1. Iterative retrieval:
   — Retrieve, generate partial answer, use partial answer to form next query, retrieve again
   — Loop continues until full answer assembled
   — Used in ReAct-style agents
2. Chain-of-thought with retrieval:
   — LLM generates reasoning steps, each step triggers a retrieval
   — More controlled than free-form iteration
3. Graph traversal (Graph RAG):
   — Follow entity relationships in knowledge graph
   — Natural fit for multi-hop entity reasoning

```
# Iterative multi-hop retrieval
async def multi_hop_retrieve(initial_query: str, max_hops: int = 3) -> str:
    context_so_far = []
    current_query = initial_query

    for hop in range(max_hops):
        # Retrieve for current query
        chunks = await retrieve(current_query, k=3)
        context_so_far.extend(chunks)

        # Ask LLM: can we answer yet? If not, what do we still need?
        assessment = await llm.ainvoke(f"""
Context so far: {context_so_far}
Original question: {initial_query}

Can you fully answer the question with the above context?
If YES, answer it.
If NO, what specific information are you still missing?
""")

        if "YES" in assessment:
            return assessment  # done

        # Extract what's still needed → new query for next hop
        current_query = extract_missing_info(assessment)

    return await generate_final_answer(initial_query, context_so_far)
```

↳ Follow-up: What is the main challenge with multi-hop retrieval?

Error propagation — if the first hop retrieves the wrong document, subsequent hops are built on wrong information, compounding the error. Each hop also adds latency. Mitigation: validate intermediate results, use confidence scores, limit max hops, add explicit verification steps.

Q22. What is agentic RAG?

Frequently Asked

Agentic RAG = RAG where the retrieval strategy itself is determined by an LLM agent, which can decide WHAT to retrieve, WHEN to retrieve, and HOW MANY retrieval steps to take.
Standard RAG: fixed pipeline — always retrieve top-K, always use same query, one shot.
Agentic RAG: flexible pipeline — agent decides retrieval strategy dynamically.
Agentic RAG capabilities:
— Multi-step retrieval: retrieve, assess, decide whether more retrieval needed
— Query rewriting: agent reformulates query if first retrieval was insufficient
— Tool selection: agent chooses between vector search, SQL, web search, API calls
— Adaptive K: retrieve more chunks for complex questions, fewer for simple ones
— Self-correction: agent detects retrieval failure and tries different strategy
Frameworks:
— LlamaIndex agents with retrieval tools
— LangGraph with RAG nodes
— AutoGen with RAG capabilities
Tradeoffs:
— More accurate: adapts to query complexity
— Less predictable: harder to debug, latency varies
— More expensive: multiple LLM calls per query

↳ Follow-up: What is the difference between agentic RAG and standard RAG + ReAct?

They're essentially the same thing. ReAct is the prompting strategy that enables agentic behaviour (alternate reasoning and acting). Agentic RAG = applying ReAct where the available "actions" include retrieval tools. The term "agentic RAG" emphasises that the RAG system has agency over its retrieval strategy, not just a fixed retrieve-then-generate loop.

Q23. What is semantic caching?

Moderately Asked

Semantic caching = caching LLM responses not by exact query string match but by semantic similarity. Queries that MEAN the same thing return the cached answer without hitting the LLM.
Traditional cache: "What is the leave policy?" → cached. "What's our leave policy?" → MISS (different string).
Semantic cache: both queries have similar embeddings → HIT.
How it works:
1. Query arrives
2. Embed query → vector
3. Search cache index for similar vectors (cosine similarity > threshold)
4. If similar query found in cache → return cached answer (skip LLM)
5. If not found → call LLM → cache (query vector, answer)
Why it matters:
— LLM calls are expensive ($0.01-0.03 per 1K tokens)
— Many users ask similar questions (especially on support/knowledge base systems)
— Cache hits can save 40-60% of LLM costs in high-traffic systems
— Reduces latency for cached queries from 2-5s to <50ms
Tools: GPTCache, Langchain Semantic Cache, Redis with vector similarity

↳ Follow-up: What is the cache invalidation problem in semantic caching?

When source documents change, cached answers may become stale. A cached answer "leave policy is 20 days" is wrong if the policy changed to 25 days. Solution: cache entries should include document version or hash. When documents are re-indexed, invalidate cache entries linked to changed documents. This is complex to implement correctly — it's the hardest part of semantic caching.

Q24. Embedding drift problems?

Frequently Asked

Embedding drift = when the same text produces different embedding vectors because the embedding model was updated. This makes old stored vectors incompatible with new query vectors.
Why it's a problem:
— You store 1 million chunks with model v1 embeddings
— Model upgrades to v2 (better quality)

— Query embeddings now from v2, stored chunks from v1
— Cosine similarity between v1 and v2 embeddings is meaningless → retrieval breaks completely
Types of drift:
1. Model version drift: embedding model updated by provider (OpenAI, Cohere)
2. Fine-tune drift: you fine-tune the embedding model on new data
3. Domain drift: model performs worse as your domain data drifts from training data
Mitigations:
— Pin embedding model version: always use text-embedding-3-small@v1, never "latest"
— Track embedding model version in vector DB metadata
— When upgrading: full re-index of entire corpus with new model (expensive but necessary)
— Canary deployment: test new model on subset before full migration

⚠️ **Interview trap:** *Never use 'latest' version of an embedding model in production. Pin to a specific version. An automatic model upgrade will silently break your entire retrieval system overnight.*

↳ Follow-up: How do you migrate from one embedding model to another in production?

Blue-green migration: (1) Create new collection in vector DB. (2) Re-index all documents with new embedding model into new collection. (3) Run A/B test: route some traffic to new collection, compare retrieval quality metrics. (4) When satisfied, cut over all traffic to new collection. (5) Delete old collection. Zero downtime, safe rollback if new model is worse.

Q25. Cross-encoder vs bi-encoder reranking?

Frequently Asked

Two architectures for computing relevance between a query and a document:
Bi-encoder:
— Encodes query and document SEPARATELY into vectors
— Relevance score = cosine similarity between vectors
— Fast: document vectors pre-computed and cached
— Suitable for initial retrieval (can search millions of docs)
— Limitation: query and document encoded independently → less accurate (can't model interactions)
Cross-encoder:
— Takes (query, document) PAIR as input to a single model
— Model attends to BOTH simultaneously, outputs relevance score
— Slow: must run model for every (query, document) pair — can't pre-compute
— Much more accurate: model sees query-document interactions
— Suitable for reranking small candidate sets (20-50 docs)
The two-stage pipeline:
Stage 1 (bi-encoder): retrieve top-50 candidates fast
Stage 2 (cross-encoder): rerank top-50 to top-5 accurately

```
# Two-stage: bi-encoder retrieval → cross-encoder reranking
from sentence_transformers import SentenceTransformer, CrossEncoder

# Stage 1: Bi-encoder for fast retrieval
bi_encoder = SentenceTransformer("BAAI/bge-large-en-v1.5")

query = "What is the parental leave policy?"
query_embedding = bi_encoder.encode(query)

# Fast ANN search → top 50 candidates
candidates = vector_db.search(query_embedding, top_k=50)

# Stage 2: Cross-encoder for accurate reranking
cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

# Score each (query, candidate) pair
pairs = [(query, c.text) for c in candidates]
scores = cross_encoder.predict(pairs)

# Sort by cross-encoder score, take top 5
reranked = sorted(zip(candidates, scores),
                  key=lambda x: x[1], reverse=True)[:5]

final_chunks = [r[0] for r in reranked]
```

↳ Follow-up: Why not use cross-encoder for initial retrieval?

Cross-encoder cannot pre-compute document representations — it needs the query at inference time. So for 1 million documents and 1 query, you'd need to run the model 1 million times. At ~50ms per run, that's 14 hours per query. Completely infeasible. Bi-encoders pre-compute all document embeddings once, then similarity search takes milliseconds.

Q26. Explain retrieval latency bottlenecks.

Moderately Asked

RAG adds latency on top of LLM generation. Understanding where that time goes is critical for production optimisation.

Latency components in a RAG query:

1. Query embedding: 20-100ms (calling embedding API or local model)
2. Vector search: 5-50ms (ANN search in vector DB)
3. Metadata filtering: adds 5-20ms (pre/post filter on large collections)
4. Chunk fetch: 5-30ms (fetching chunk text by ID from storage)
5. Reranking: 50-300ms (cross-encoder, if used)
6. Context assembly: 5-10ms (building the final prompt)
7. LLM generation: 500ms-5s (the dominant cost)

Optimisations:

— Parallel embedding + prefetch: embed query while prefetching likely chunks
— Cache query embeddings: same query → skip embedding step
— In-memory vector index: keep hot data in RAM (HNSW in memory vs disk)
— Reduce reranker model size: MiniLM reranker vs large reranker (10x speed difference)
— Streaming generation: start returning tokens before full generation completes

↳ Follow-up: What is the P99 latency target for production RAG systems?

Industry benchmark: total RAG response P50 < 2s, P99 < 5s for chat applications. For real-time autocomplete or search suggestions: < 200ms total (must skip reranking, use approximate retrieval, semantic cache). Profile your system and identify the biggest bottleneck — usually LLM generation (70% of total time), not retrieval (10-20%).

Q27. Hybrid retrieval architecture design?

Frequently Asked

Production hybrid retrieval architecture combines dense + sparse retrieval with reranking for maximum quality.

Full architecture:

1. Query processing: query rewriting/expansion (optional)
2. Parallel retrieval:
   — Dense (vector): top-50 from vector DB using bi-encoder embeddings
   — Sparse (BM25): top-50 from Elasticsearch/OpenSearch using keyword matching
3. Fusion: Reciprocal Rank Fusion (RRF) merges two lists → unified top-100
4. Metadata filtering: apply filters (date, department, access level) on fused results
5. Reranking: cross-encoder reranks top-100 → top-10
6. Contextual compression: extract only relevant sentences from top-10
7. Context assembly: build final prompt with top-5 chunks
8. LLM generation

Infrastructure:

— Vector DB: Qdrant or Weaviate (native hybrid search support)
— Sparse index: Elasticsearch/OpenSearch (or Qdrant's sparse vectors)
— Reranker: Cohere Rerank API or BGE-Reranker
— Embedding model: text-embedding-3-small or BGE-Large

```
# Hybrid retrieval with RRF fusion
def reciprocal_rank_fusion(
    dense_results: list,
    sparse_results: list,
    k: int = 60
) -> list:
    scores = {}
    for rank, doc in enumerate(dense_results):
        doc_id = doc.id
        scores[doc_id] = scores.get(doc_id, 0) + 1 / (rank + k)
```

```
    for rank, doc in enumerate(sparse_results):
        doc_id = doc.id
        scores[doc_id] = scores.get(doc_id, 0) + 1 / (rank + k)

    # Sort by fused score
    sorted_ids = sorted(scores.items(), key=lambda x: x[1], reverse=True)
    return [get_doc(doc_id) for doc_id, _ in sorted_ids[:50]]

# Full hybrid pipeline
async def hybrid_retrieve(query: str, top_k: int = 5) -> list:
    dense_results, sparse_results = await asyncio.gather(
        dense_search(query, k=50),
        bm25_search(query, k=50),
    )
    fused = reciprocal_rank_fusion(dense_results, sparse_results)
    reranked = reranker.rerank(query, fused, top_n=top_k)
    return reranked
```

⭐ **Key Insight:** Dense + BM25 + RRF + Cross-encoder reranker is the gold standard production RAG retrieval stack. Each layer adds quality with acceptable latency tradeoff.

Q28. RAG evaluation metrics?

Frequently Asked

Complete list of RAG evaluation metrics:
RETRIEVAL METRICS:
— Context Precision: fraction of retrieved chunks that are actually relevant. High precision = less noise for LLM.
— Context Recall: fraction of all relevant information that was retrieved. High recall = LLM has what it needs.
— Context Relevance: similar to precision — are retrieved chunks relevant to the query?
— MRR (Mean Reciprocal Rank): 1/rank of first relevant result. Measures how highly relevant docs are ranked.
— Hit Rate / Recall@K: does the correct chunk appear in top-K? (K=1,3,5,10)
— NDCG (Normalised Discounted Cumulative Gain): comprehensive ranked retrieval quality metric.
GENERATION METRICS:
— Faithfulness: is the generated answer supported by retrieved context? Detects hallucination within RAG.
— Answer Relevance: does the answer address what was asked?
— Answer Correctness: is the answer factually correct vs ground truth?
— Answer Completeness: does the answer cover all aspects of the question?
SYSTEM METRICS:
— End-to-end latency: total response time
— Cost per query: embedding + retrieval + reranking + LLM generation costs
— Throughput: queries per second

↳ Follow-up: Which metric should you optimise first?

Context Recall first — if you're not retrieving the right information, nothing else matters. Then Context Precision (reduce noise). Then Faithfulness (ensure LLM uses what's retrieved). Then Answer Relevance. This order reflects the data flow: retrieval feeds generation. Fix retrieval before tuning generation prompts.

Q29. What is lost-in-the-middle?

Frequently Asked

Lost-in-the-middle (Liu et al., 2023) is the empirically documented phenomenon where LLMs perform significantly worse when the relevant information is positioned in the MIDDLE of a long context, even when it fits within the context window.
The finding: in controlled experiments, LLM performance was highest when relevant info was at the START (primacy) or END (recency) of context, and significantly lower when placed in the middle.
Why it happens: LLM attention mechanisms are biased toward the beginning and end of sequences (positional attention patterns). The middle of long sequences receives relatively less attention weight.
Implications for RAG:
— Don't blindly concatenate retrieved chunks in retrieval order
— Put most relevant chunks FIRST and LAST in the context
— If using many chunks (> 5), quality degrades for info in the middle
— Prefer fewer, higher-quality chunks over many mediocre ones

Mitigations:
1. Reranking: ensure top-ranked chunks appear first/last
2. Limit chunk count: use 3-5 high-quality chunks, not 10 mediocre ones
3. Contextual compression: compress chunks to only relevant sentences
4. "Positional injection": explicitly repeat key facts at the end of context

⚠️ **Interview trap:** *This is NOT just a theoretical concern. In production RAG systems with 10+ retrieved chunks, lost-in-the-middle is a measurable quality problem. Limit your chunks to top-5 with reranking.*

↳ Follow-up: Does lost-in-the-middle apply equally to all models?

No — the severity varies by model. GPT-4 and Claude 3.5 are better at attending to the middle than older models, but the effect still exists. Gemini 1.5 claims reduced lost-in-the-middle with their long-context architecture. In practice: always test your specific model and never assume this problem doesn't apply.

Q30. Long-context models vs RAG tradeoffs?

Frequently Asked

This is the core architectural debate in production AI systems (2024-2025):
LONG-CONTEXT WINS:
— Holistic understanding: model can cross-reference multiple sections, understand document structure
— No retrieval errors: can't miss information if everything is in context
— Simpler architecture: no vector DB, no chunking, no embedding pipeline
— Better for: annual reports, legal contracts, codebases (where full context matters)
RAG WINS:
— Scale: retrieve from 1M+ documents — impossible with any context window
— Cost: 5 retrieved chunks (1K tokens) vs full corpus (200K tokens) = 200x cost difference
— Freshness: add new documents instantly, no re-training
— Access control: different users retrieve different document subsets
— Citations: can trace exactly which chunk the answer came from
— Latency: smaller context = faster generation
Hybrid approach (best of both):
— RAG for wide retrieval from large corpus
— Long context to process the retrieved set holistically
— Typical: retrieve 10-20 chunks → pass to model with 8K context window
The verdict: RAG remains dominant for enterprise knowledge systems. Long context is a complement, not a replacement.

⭐ **Key Insight:** Long-context models didn't kill RAG — they made RAG more powerful by being able to process larger retrieved contexts more accurately.

↳ Follow-up: What is the cost difference between 200K long-context and RAG at scale?

Example: 200K token context vs 2K token RAG context (5 × 400-token chunks). At GPT-4o pricing ($0.005/1K input tokens): Long context = 200K × $0.005/1K = $1.00 per query. RAG = 2K × $0.005/1K = $0.01 per query. That's 100x cost difference per query. At 1M queries/day: Long context = $1M/day. RAG = $10K/day. The economics strongly favour RAG at scale.